

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №3

з дисципліни «Аналіз та рефакторинг коду»

на тему «Розробка бізнес логіки та функцій адміністрування серверної
частини системи»

Виконав:
ст. гр. ПЗП-23-8
Потурайко Н.І.

Прийняв:
асистент. каф. ПІ.
Дашенков Д. С.

Харків 2025

3. 1 Мета роботи

Метою лабораторної роботи є проєктування та програмна реалізація складної бізнес-логіки та функцій адміністрування для серверної частини системи моніторингу мікроклімату «Aquila». Робота передбачає розробку алгоритмів автоматичної обробки телеметричних даних з реалізацією механізмів фільтрації (debounce) та генерації тривоги, забезпечення цілісності даних при каскадних операціях за допомогою ACID-транзакцій, а також впровадження підсистеми адміністрування. Остання включає рольову модель доступу, ведення журналу аудиту подій, блокування користувачів та експорт аналітичних даних. Важливою складовою роботи є візуалізація алгоритмів та взаємодії компонентів системи за допомогою UML-діаграм діяльності (Activity Diagram) та послідовності (Sequence Diagram).

3.2 Опис інженерних рішень та алгоритмів бізнес-логіки

Реалізація бізнес-логіки серверної частини системи Aquila базується на подійно-орієнтованій архітектурі в середовищі Node.js. Під час розробки основний акцент було зроблено на забезпеченні надійності даних, оптимізації навантаження на базу даних та безпеці адміністративних операцій.

Ключовим алгоритмом системи є комплексна обробка телеметричних даних, яка починається з ідентифікації та валідації джерела інформації. Система перевіряє існування пристрою за його MAC-адресою та оновлює часову мітку останньої активності. Наступним етапом є аналіз порогових значень, під час якого отримані показники температури, вологості та рівня шуму порівнюються із заданими критичними межами. Для запобігання дублюванню сповіщень та переповнення бази даних реалізовано логіку фільтрації тривоги. Перед реєстрацією нового інциденту в базі даних система виконує пошук аналогічних активних тривоги для цього пристрою, створених протягом визначеного інтервалу часу, і створює новий запис лише за відсутності нещодавніх дублікатів.

Для забезпечення цілісності даних при видаленні сутностей верхнього рівня ієрархії, таких як кімнати, застосовано механізм ACID-транзакцій. Операція видалення виконується атомарно, гарантуючи, що всі залежні дані будуть оброблені коректно. Процедура передбачає послідовне видалення пов'язаних тривоги, історії вимірювань, збережених зображень та самих пристроїв, і лише після успішного виконання цих дій

видаляється запис про кімнату. У разі виникнення будь-якої помилки під час цього процесу система автоматично ініціює повний відкат змін, що унеможливорює порушення зв'язності даних.

Підсистема адміністрування функціонує на основі ролівої моделі доступу, що забезпечується спеціальним проміжним програмним забезпеченням для перевірки прав користувачів перед виконанням критичних запитів. З метою підвищення безпеки та забезпечення підзвітності усі значущі дії адміністратора, такі як блокування користувачів або зміна конфігурації, фіксуються в спеціалізованому журналі аудиту подій.

3.3 Програмна реалізація бізнес-логіки та функцій адміністрування

Програмна реалізація серверної частини виконана мовою JavaScript з використанням платформи Node.js. Взаємодія з базою даних реалізована через драйвер pg, що дозволяє виконувати параметризовані SQL-запити для захисту від ін'єкцій. Нижче наведено ключові фрагменти коду, що демонструють реалізацію алгоритмів бізнес-логіки.

```
app.post('/api/telemetry', async (req, res) => {
  const { macAddress, temperature, humidity, lightLevel,
    soundLevel } = req.body;

  try {
    const deviceRes = await pool.query('SELECT id FROM devices
WHERE mac_address = $1', [macAddress]);
    if (deviceRes.rows.length === 0) return res.status(404).json({
error: 'Device not found' });

    const deviceId = deviceRes.rows[0].id;
    await pool.query('UPDATE devices SET last_seen = NOW() WHERE
id = $1', [deviceId]);

    const insertRes = await pool.query(
      'INSERT INTO measurements (device_id, temperature, humidity,
light_level, sound_level) VALUES ($1, $2, $3, $4, $5) RETURNING
*',
      [deviceId, temperature, humidity, lightLevel, soundLevel]
    );

    const TEMP_MAX = 28;
    if (temperature > TEMP_MAX) {
      const recentAlert = await pool.query(`
        SELECT id FROM alerts
```

```

        WHERE device_id = $1 AND type = 'TEMPERATURE'
        AND created_at > NOW() - INTERVAL '15 minutes'
    `, [deviceId]);

    if (recentAlert.rows.length === 0) {
        await pool.query(
            'INSERT INTO alerts (device_id, type, message) VALUES
($1, $2, $3)',
            [deviceId, 'TEMPERATURE', `⚠ Температура критична:
${temperature}°C`]
        );
    }
}
res.status(201).json({ status: 'success', data:
insertRes.rows[0] });
} catch (err) {
    res.status(500).json({ error: err.message });
}
});

```

Для забезпечення цілісності даних при видаленні кімнат застосовано транзакційний підхід. Це гарантує, що при видаленні кімнати будуть коректно видалені всі пов'язані з нею сутності.

```

app.delete('/api/rooms/:id', async (req, res) => {
    const { id } = req.params;
    const client = await pool.connect();

    try {
        await client.query('BEGIN');

        const deleteQuery = 'DELETE FROM %I WHERE device_id IN (SELECT
id FROM devices WHERE room_id = $1)';
        await client.query(format(deleteQuery, 'alerts'), [id]);
        await client.query(format(deleteQuery, 'measurements'), [id]);
        await client.query(format(deleteQuery, 'snapshots'), [id]);

        await client.query('DELETE FROM devices WHERE room_id = $1',
[id]);
        const result = await client.query('DELETE FROM rooms WHERE id
= $1 RETURNING *', [id]);

        await client.query('COMMIT'); // Фіксація змін
        res.json({ message: 'Room deleted successfully', room:
result.rows[0] });
    } catch (err) {

```

```

    await client.query('ROLLBACK'); // Відкат змін у разі помилки
    res.status(500).json({ error: err.message });
  } finally {
    client.release();
  }
});

```

Підсистема адміністрування захищена спеціальним проміжним обробником (middleware), який перевіряє роль користувача перед наданням доступу до функцій керування та експорту даних.

```

const isAdmin = async (req, res, next) => {
  const userId = req.headers['x-user-id'];
  if (!userId) return res.status(401).json({ error: 'Unauthorized' });

  const result = await pool.query('SELECT role FROM users WHERE id = $1', [userId]);
  if (result.rows.length === 0 || result.rows[0].role !== 'admin') {
    return res.status(403).json({ error: 'Access denied: Admins only' });
  }
  next();
};

app.put('/api/admin/users/:id/block', isAdmin, async (req, res) => {
  const { id } = req.params;
  const { block } = req.body;

  try {
    const result = await pool.query(
      'UPDATE users SET is_blocked = $1 WHERE id = $2 RETURNING id, email',
      [block, id]
    );

    await pool.query(
      'INSERT INTO audit_logs (action, details) VALUES ($1, $2)',
      ['USER_BLOCK_CHANGE', `User ${result.rows[0].email} blocked status: ${block}`]
    );

    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

    }
  });

```

3.4 Графічні ілюстрації

Для візуалізації розроблених алгоритмів та процесів взаємодії компонентів системи було побудовано дві UML діаграми.

Діаграма діяльності (Activity Diagram) деталізує алгоритм обробки вхідних телеметричних даних на стороні сервера. На схемі відображено логічні розгалуження процесу: валідацію вхідного запиту, перевірку існування пристрою, аналіз температурних показників та логіку фільтрації дублікатів тривог (Debounce). Цей алгоритм забезпечує захист бази даних від надлишкової інформації та гарантує реакцію лише на значущі події.

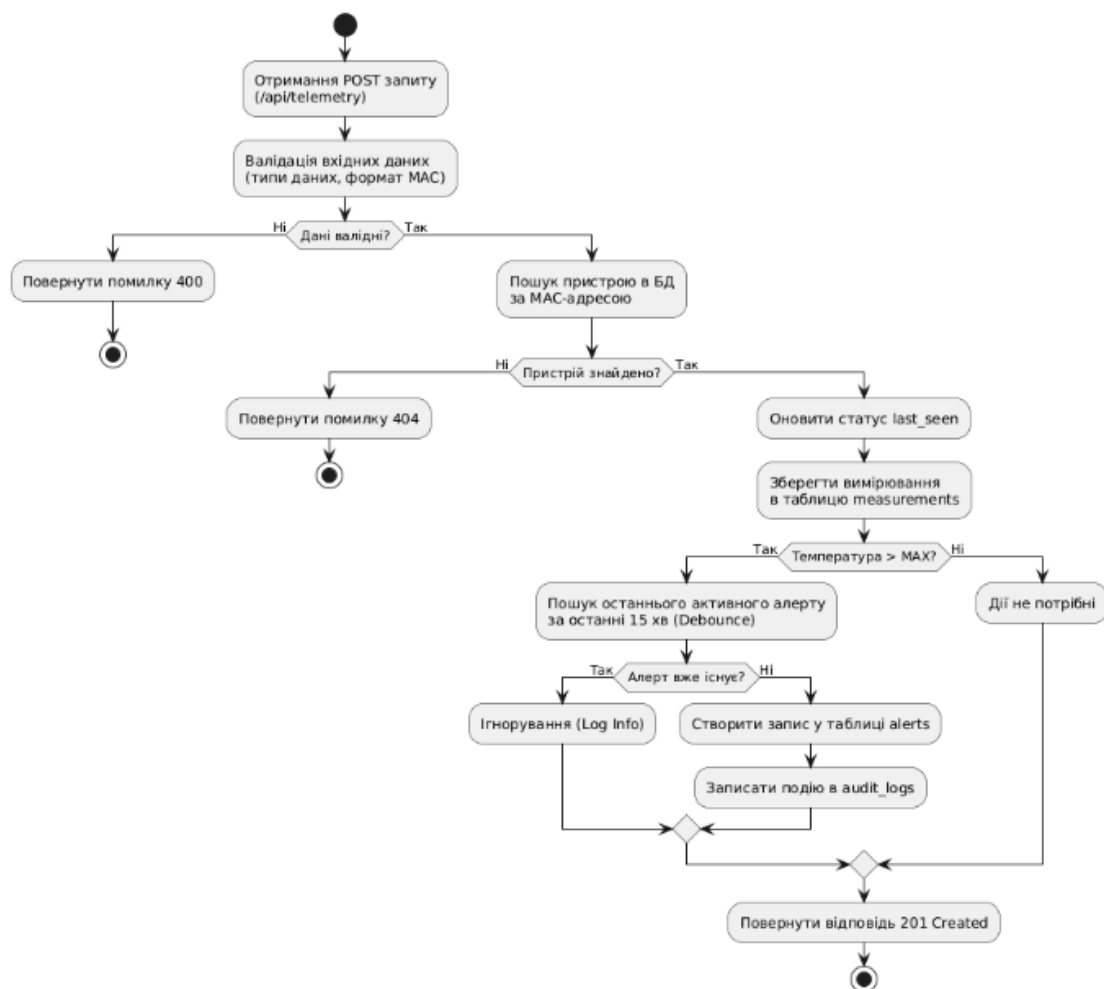


Рисунок 3.1 – UML діаграма діяльності

Діаграма послідовності (Sequence Diagram) ілюструє повний цикл обміну даними в системі: від відправки показників фізичним сенсором до

отримання звіту адміністратором. На діаграмі показано життєвий цикл запиту, включаючи перевірку прав доступу (Middleware) при зверненні адміністратора до захищених ресурсів та взаємодію сервера з базою даних PostgreSQL.

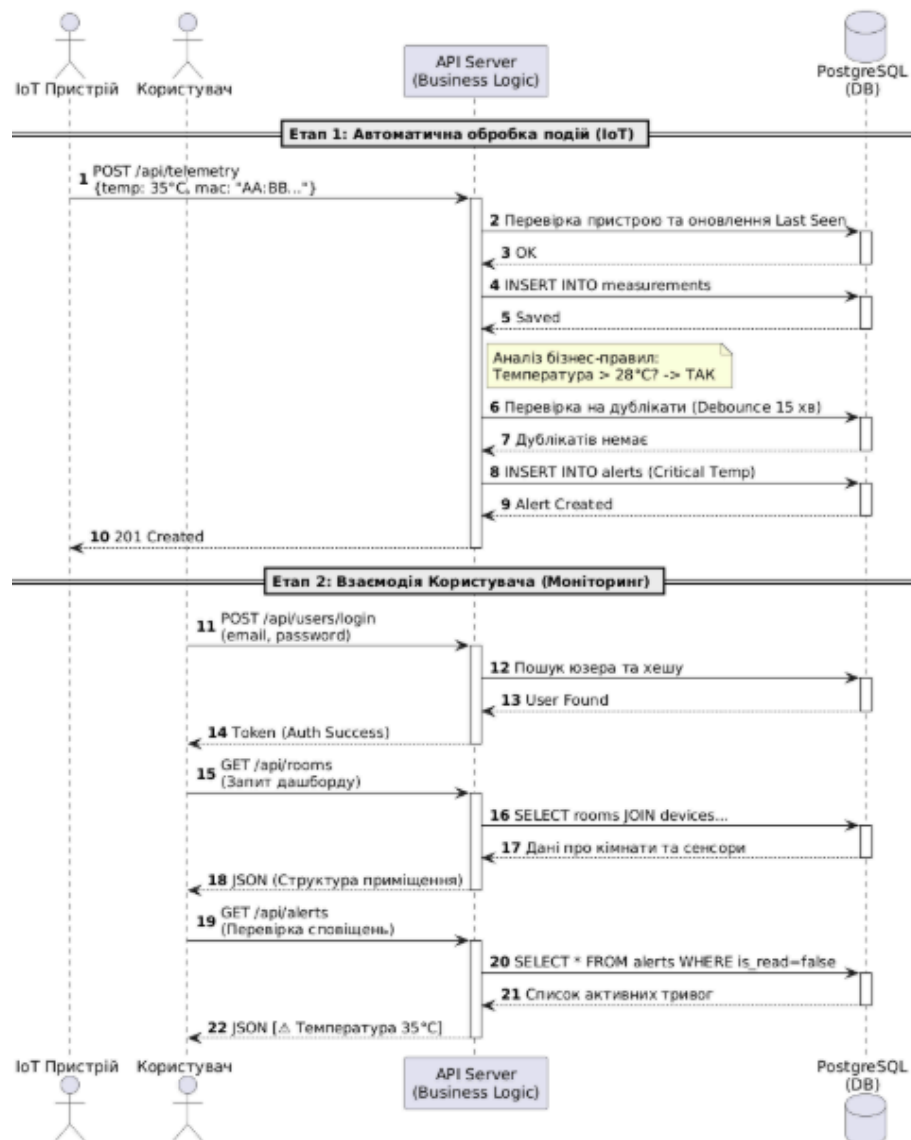


Рисунок 3.2 – UML діаграма послідовності (Взаємодія компонентів системи)

3.5 Висновки

У ході виконання лабораторної роботи було успішно спроектовано та програмно реалізовано комплексні алгоритми бізнес-логіки та підсистему адміністрування для серверної частини платформи Aquila. Основну увагу було приділено розробці інтелектуальних механізмів обробки телеметричних даних, зокрема впроваджено алгоритм дебаунсу для

фільтрації надлишкових сповіщень та запобігання переповненню бази даних дубльованими записами про критичні події. Для забезпечення надійності та узгодженості даних при виконанні деструктивних операцій, таких як видалення кімнат разом із пристроями та історією вимірювань, було застосовано механізм ACID-транзакцій, що гарантує атомарність змін. Окрім того, систему доповнено засобами адміністрування, включаючи рольову модель розмежування доступу, ведення журналу аудиту для фіксації дій адміністраторів та функціонал експорту аналітичних даних. Розроблені UML-діаграми діяльності та послідовності наочно відображають алгоритми роботи системи та повний цикл руху даних від фізичних сенсорів до кінцевого інтерфейсу користувача, підтверджуючи працездатність та архітектурну завершеність серверної частини.